

Date:

Exp No:9

Aim:Implement page replacement algorithms:FIFO,LRU and Optimal

Theory

1. FIFO (First-In-First-Out) Algorithm

Theory

Definition

FIFO replaces the page that has been in memory for the **longest time**.

It works like a **queue**:

- The page that enters first
- Leaves first

Working

1. Maintain a queue of pages in memory.
2. When a new page arrives:
 - If it is already in memory → Page Hit.
 - If not → Page Fault.
3. If frames are full:
 - Remove the oldest page (first entered).
 - Insert the new page.
4. Count total page faults.

1. Write the c program OS_exp9_1.c

```
#include <stdio.h>
```

```
int main() {  
    int frames[10], pages[50];  
    int frameCount, pageCount;  
    int i, j, k;  
    int pageFaults = 0;  
    int index = 0; // FIFO pointer  
    int hit;  
  
    printf("Enter number of frames: ");  
    scanf("%d", &frameCount);  
  
    printf("Enter number of pages: ");  
    scanf("%d", &pageCount);
```

```

printf("Enter page reference string:\n");
for(i = 0; i < pageCount; i++) {
    scanf("%d", &pages[i]);
}

// Initialize frames as empty
for(i = 0; i < frameCount; i++) {
    frames[i] = -1;
}

printf("\nStep\tPage\tFrames\t\tStatus\n");
printf("-----\n");

for(i = 0; i < pageCount; i++) {
    hit = 0;

    // Check for page hit
    for(j = 0; j < frameCount; j++) {
        if(frames[j] == pages[i]) {
            hit = 1;
            break;
        }
    }

    // If page fault
    if(hit == 0) {
        frames[index] = pages[i];
        index = (index + 1) % frameCount;
        pageFaults++;
    }

    // Print step details
    printf("%d\t%d\t", i+1, pages[i]);
    for(k = 0; k < frameCount; k++) {
        if(frames[k] != -1)
            printf("%d ", frames[k]);
        else
            printf("- ");
    }

    if(hit)
        printf("\tHit\n");
    else
        printf("\tFault\n");
}

printf("\nTotal Page Faults = %d\n", pageFaults);
printf("Total Page Hits = %d\n", pageCount - pageFaults);

return 0;
}

```

2.Compile and Run

```
gcc OS_exp9_1.c -o OS_exp9_1
./ OS_exp9_1
```

3.Expected output

```
Enter number of frames: 3
Enter number of pages: 12
Enter page reference string:
1 2 0 3 4 2 3 7 8 2 4 6

Step   Page   Frames      Status
-----
1       1       1 - -      Fault
2       2       1 2 -      Fault
3       0       1 2 0      Fault
4       3       3 2 0      Fault
5       4       3 4 0      Fault
6       2       3 4 2      Fault
7       3       3 4 2      Hit
8       7       7 4 2      Fault
9       8       7 8 2      Fault
10      2       7 8 2      Hit
11      4       7 8 4      Fault
12      6       6 8 4      Fault

Total Page Faults = 10
Total Page Hits = 2
```

2.LRU (Least Recently Used) Algorithm

Theory

Definition

LRU replaces the page that has **not been used for the longest period of time**.

It assumes that pages used recently will likely be used again soon.

Working

1. Maintain frames to store pages.
2. For each page reference:
 - If page is already in frame → Page Hit.
 - Else → Page Fault.
3. If frames are full:
 - Replace the page that was least recently used.
4. Count total page faults.

1. Write the c program OS_exp9_2.c

```
#include <stdio.h>

int main() {
    int frames[10], pages[50], time[10];
    int frameCount, pageCount;
    int i, j, k;
    int pageFaults = 0;
    int counter = 0;
    int hit, pos, min;

    printf("Enter number of frames: ");
    scanf("%d", &frameCount);

    printf("Enter number of pages: ");
    scanf("%d", &pageCount);

    printf("Enter page reference string:\n");
    for(i = 0; i < pageCount; i++) {
        scanf("%d", &pages[i]);
    }

    // Initialize frames
    for(i = 0; i < frameCount; i++) {
        frames[i] = -1;
        time[i] = 0;
    }

    printf("\nStep\tPage\tFrames\t\tStatus\n");
    printf("-----\n");

    for(i = 0; i < pageCount; i++) {
        hit = 0;

        // Check for page hit
        for(j = 0; j < frameCount; j++) {
            if(frames[j] == pages[i]) {
                counter++;
                time[j] = counter;
                hit = 1;
                break;
            }
        }

        // If page fault
        if(hit == 0) {
            pos = -1;

            // Check for empty frame
```

```

for(j = 0; j < frameCount; j++) {
    if(frames[j] == -1) {
        pos = j;
        break;
    }
}

// If no empty frame, replace LRU
if(pos == -1) {
    min = time[0];
    pos = 0;

    for(j = 1; j < frameCount; j++) {
        if(time[j] < min) {
            min = time[j];
            pos = j;
        }
    }
}

counter++;
frames[pos] = pages[i];
time[pos] = counter;
pageFaults++;
}

// Print current frames
printf("%d\t%d\t", i+1, pages[i]);
for(k = 0; k < frameCount; k++) {
    if(frames[k] != -1)
        printf("%d ", frames[k]);
    else
        printf("- ");
}

if(hit)
    printf("\tHit\n");
else
    printf("\tFault\n");
}

printf("\nTotal Page Faults = %d\n", pageFaults);
printf("Total Page Hits = %d\n", pageCount - pageFaults);

return 0;
}

```

2.Compile and Run

```
gcc OS_exp9_2.c -o OS_exp9_2
./ OS_exp9_2
```

3.Expected output

```
Enter number of frames: 3
Enter number of pages: 12
Enter page reference string:
1 2 0 3 4 2 3 7 8 2 4 6

Step   Page   Frames           Status
-----
1      1      1 - -          Fault
2      2      1 2 -          Fault
3      0      1 2 0          Fault
4      3      3 2 0          Fault
5      4      3 4 0          Fault
6      2      3 4 2          Fault
7      3      3 4 2          Hit
8      7      3 7 2          Fault
9      8      3 7 8          Fault
10     2      2 7 8          Fault
11     4      2 4 8          Fault
12     6      2 4 6          Fault

Total Page Faults = 11
Total Page Hits = 1
```

3.Optimal Page Replacement Algorithm

Theory

Definition

The **Optimal algorithm** replaces the page that will **not be used for the longest time in the future**.

It looks ahead in the reference string and chooses the best possible page to remove.

Working

1. For each page reference:
 - If page is already in frame → Page Hit.
 - Else → Page Fault.
2. If frames are not full → Insert page.
3. If frames are full:
 - Look ahead in the reference string.
 - Replace the page whose next use is farthest in future.
4. Count total page faults.

1. Write the c program OS_exp9_3.c

```
#include <stdio.h>
```

```
int main() {
    int frames[10], pages[50];
    int frameCount, pageCount;
    int i, j, k;
    int pageFaults = 0;
    int hit, pos, farthest, index;

    printf("Enter number of frames: ");
    scanf("%d", &frameCount);

    printf("Enter number of pages: ");
    scanf("%d", &pageCount);

    printf("Enter page reference string:\n");
    for(i = 0; i < pageCount; i++) {
        scanf("%d", &pages[i]);
    }

    // Initialize frames
    for(i = 0; i < frameCount; i++) {
        frames[i] = -1;
    }

    printf("\nStep\tPage\tFrames\t\tStatus\n");
    printf("-----\n");

    for(i = 0; i < pageCount; i++) {
        hit = 0;

        // Check for page hit
        for(j = 0; j < frameCount; j++) {
            if(frames[j] == pages[i]) {
                hit = 1;
                break;
            }
        }

        // If page fault
        if(hit == 0) {
            // Check for empty frame
            pos = -1;
            for(j = 0; j < frameCount; j++) {
                if(frames[j] == -1) {
                    pos = j;
                    break;
                }
            }
        }
    }
}
```

```

// If no empty frame, apply Optimal replacement
if(pos == -1) {
    farthest = -1;
    for(j = 0; j < frameCount; j++) {
        int nextUse = -1;

        for(k = i + 1; k < pageCount; k++) {
            if(frames[j] == pages[k]) {
                nextUse = k;
                break;
            }
        }

        if(nextUse == -1) { // Not used again
            pos = j;
            break;
        }

        if(nextUse > farthest) {
            farthest = nextUse;
            pos = j;
        }
    }
}

frames[pos] = pages[i];
pageFaults++;
}

// Display frames
printf("%d\t%d\t", i+1, pages[i]);
for(j = 0; j < frameCount; j++) {
    if(frames[j] != -1)
        printf("%d ", frames[j]);
    else
        printf("- ");
}

if(hit)
    printf("\tHit\n");
else
    printf("\tFault\n");
}

printf("\nTotal Page Faults = %d\n", pageFaults);
printf("Total Page Hits = %d\n", pageCount - pageFaults);

return 0;
}

```


2.Compile and Run

```
gcc OS_exp9_3.c -o OS_exp9_3
./ OS_exp9_3
```

3.Expected output

```
Enter number of frames: 3
Enter number of pages: 12
Enter page reference string:
1 2 0 3 4 2 3 7 8 2 4 6
```

Step	Page	Frames	Status
1	1	1 - -	Fault
2	2	1 2 -	Fault
3	0	1 2 0	Fault
4	3	3 2 0	Fault
5	4	3 2 4	Fault
6	2	3 2 4	Hit
7	3	3 2 4	Hit
8	7	7 2 4	Fault
9	8	8 2 4	Fault
10	2	8 2 4	Hit
11	4	8 2 4	Hit
12	6	6 2 4	Fault

```
Total Page Faults = 8
Total Page Hits = 4
```